

DOCUMENTAÇÃO DA CALCULADORA DE CONVERSÃO DE BASES

Aplicação web desenvolvida com Python, Flask, HTML e CSS

Objetivo: documentar a estrutura, o funcionamento e as principais linhas dos arquivos `app.py`, `index.html` e `style.css`.

1. Visão geral do projeto

A aplicação permite converter números entre as bases binária (2), octal (8), decimal (10) e hexadecimal (16). O frontend é formado por HTML e CSS, enquanto o backend é implementado em Python com o framework Flask.

```
Calculadora-Convertao-Bases/  
|  
|-- app.py  
|-- templates/  
|   |-- index.html  
|-- static/  
|   |-- style.css
```

O navegador exibe o formulário criado em HTML. Quando o usuário pressiona "Converter", os dados são enviados ao Flask por uma requisição POST. O Python faz a conversão e devolve o resultado ao template HTML.

2. Documentação do app.py

O arquivo app.py é o núcleo do backend. Ele cria a aplicação Flask, define a função de conversão e controla a rota principal.

Importação do Flask

```
from flask import Flask, render_template, request
```

Flask cria a aplicação web; render_template carrega o HTML da pasta templates; request acessa os dados enviados pelo formulário.

Criação da aplicação

```
app = Flask(__name__)
```

Cria a instância principal da aplicação. __name__ informa ao Flask o módulo atual.

Função de conversão

```
def converter_base(valor, base_origem, base_destino):  
    numero_decimal = int(valor, base_origem)
```

Recebe o valor e as bases. int(valor, base_origem) interpreta o número na base indicada e o transforma em um inteiro decimal intermediário.

Conversão para binário

```
if base_destino == 2:  
    return bin(numero_decimal)[2:]
```

bin() gera o binário e [2:] remove o prefixo 0b.

Conversão para octal

```
elif base_destino == 8:  
    return oct(numero_decimal)[2:]
```

oct() gera o octal e [2:] remove o prefixo 0o.

Conversão para decimal

```
elif base_destino == 10:  
    return str(numero_decimal)
```

O número já está em decimal; str() o transforma em texto para exibição.

Conversão para hexadecimal

```
elif base_destino == 16:  
    return hex(numero_decimal)[2:].upper()
```

hex() gera o hexadecimal, [2:] remove 0x e upper() deixa as letras maiúsculas.

Rota principal

```
@app.route("/", methods=["GET", "POST"])  
def index():  
    resultado = ""  
    erro = ""
```

A rota / é a página inicial. GET abre a página e POST recebe o formulário. resultado e erro começam vazios.

Recebimento dos dados

```
if request.method == "POST":  
    valor = request.form.get("valor")  
    base_origem = int(request.form.get("base_origem"))  
    base_destino = int(request.form.get("base_destino"))
```

Recupera o número e as bases selecionadas. As bases são convertidas para inteiro porque os formulários enviam texto.

Tratamento de erro

```
try:  
    resultado = converter_base(valor, base_origem, base_destino)  
except ValueError:  
    erro = "Valor inválido para a base selecionada."
```

Tenta converter o número. Se houver caracteres incompatíveis com a base escolhida, ValueError é tratado e uma mensagem é preparada.

Renderização

```
return render_template(  
    "index.html",  
    resultado=resultado,  
    erro=erro  
)
```

Carrega index.html e envia resultado e erro para o template Jinja.

Servidor

```
if __name__ == "__main__":  
    app.run(debug=True)
```

Inicia o servidor quando app.py é executado diretamente. debug=True facilita o desenvolvimento e os testes.

3. Documentação do index.html

O arquivo templates/index.html define a estrutura da interface exibida no navegador e contém expressões Jinja utilizadas pelo Flask.

Estrutura inicial

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Conversor de Bases</title>
</head>
```

Define HTML5, idioma, codificação UTF-8, adaptação à tela e título da aba.

Ligação com o CSS

```
<link rel="stylesheet"
      href="{{ url_for('static', filename='style.css') }}">
```

Importa static/style.css. url_for é processado pelo Flask/Jinja.

Apresentação

```
<div class="container">
  <h1>Conversor de Bases</h1>
  <p class="descricao">Converte números entre binário, decimal, octal e hexadecimal.</p>
```

Cria o contêiner principal, título e descrição.

Formulário

```
<form method="POST">
  <label for="valor">Número</label>
  <input type="text" id="valor" name="valor"
        placeholder="Digite o número" required>
```

POST envia os dados ao backend. name='valor' corresponde ao nome lido por request.form no Python.

Base de origem

```
<select id="base_origem" name="base_origem">
  <option value="2">Binário</option>
  <option value="8">Octal</option>
  <option value="10">Decimal</option>
  <option value="16">Hexadecimal</option>
</select>
```

Permite escolher a base do número informado. Os valores numéricos são enviados ao backend.

Base de destino

```
<select id="base_destino" name="base_destino">
  <option value="10">Decimal</option>
  <option value="2">Binário</option>
  <option value="8">Octal</option>
  <option value="16">Hexadecimal</option>
</select>
```

Permite escolher a base para a qual o valor será convertido.

Botão

```
<button type="submit">Converter</button>
```

Envia o formulário ao Flask.

Resultado com Jinja

```
{% if resultado %}
<div class="resultado">
  <p>Resultado</p>
  <h2>{{ resultado }}</h2>
</div>
{% endif %}
```

O bloco só aparece quando existe resultado. {{ resultado }} insere o valor calculado pelo Python.

Erro com Jinja

```
{% if erro %}
<div class="erro">{{ erro }}</div>
{% endif %}
```

Exibe a mensagem preparada pelo backend quando ocorre uma entrada inválida.

4. Documentação do style.css

O arquivo static/style.css controla exclusivamente a aparência visual da aplicação.

Reset geral

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

Remove espaçamentos padrão e padroniza o cálculo das dimensões.

Corpo da página

```
body {
  font-family: Arial, sans-serif;
  background: linear-gradient(135deg, #0f172a, #1e293b);
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  color: white;
}
```

Define fonte, fundo, altura e centralização da calculadora com Flexbox.

Container

```
.container {
  width: 500px;
  background: #111827;
  padding: 40px;
  border-radius: 16px;
  box-shadow: 0 20px 50px rgba(0,0,0,0.4);
}
```

Cria a caixa principal com largura, fundo, espaçamento, cantos arredondados e sombra.

Campos

```
input,
select {
  width: 100%;
  padding: 13px;
  border-radius: 8px;
  border: 1px solid #334155;
  background: #1e293b;
  color: white;
  font-size: 16px;
  outline: none;
}
```

Padroniza a aparência dos campos de texto e seleção.

Bases

```
.bases {
  display: flex;
  align-items: end;
  gap: 15px;
  margin-top: 20px;
  margin-bottom: 25px;
}

.bases div {
  flex: 1;
}
```

Coloca as duas seleções lado a lado e distribui o espaço igualmente.

Botão

```
button {
  width: 100%;
  padding: 14px;
  border: none;
  border-radius: 8px;
  background: #0ea5e9;
  color: white;
  font-size: 17px;
  font-weight: bold;
  cursor: pointer;
  transition: 0.2s;
}
```

Define dimensões, cor, tipografia e comportamento visual do botão.

Hover

```
button:hover {
  background: #0284c7;
  transform: translateY(-2px);
}
```

Altera a aparência do botão quando o ponteiro passa sobre ele.

Resultado

```
.resultado {
  margin-top: 30px;
  padding: 20px;
  text-align: center;
  background: #1e293b;
  border-radius: 10px;
  border: 1px solid #334155;
}
```

Cria a caixa visual onde o resultado é apresentado.

Erro

```
.erro {
  margin-top: 20px;
  padding: 15px;
  border-radius: 8px;
  background: #f1d1d1;
  color: #fecaca;
  text-align: center;
}
```

Destaca mensagens de erro em tons de vermelho.

5. Fluxo de funcionamento

1. O usuário acessa a página.
2. Flask renderiza templates/index.html.
3. O usuário informa um número e escolhe as bases.
4. O formulário é enviado por POST.
5. app.py recebe os valores.
6. converter_base() converte primeiro para decimal.
7. O valor é convertido para a base de destino.
8. Flask devolve o resultado ao HTML.
9. O Jinja apresenta o resultado na página.

6. Exemplo prático

Entrada: 11111111 em binário; destino: hexadecimal. O backend executa `int('11111111', 2)` e obtém 255. Depois executa `hex(255)[2:].upper()` e obtém FF.

7. Tecnologias utilizadas

Python: lógica de conversão.

Flask: servidor web e comunicação entre frontend e backend.

HTML5: estrutura da interface.

CSS3: aparência da interface.

Jinja: inserção de dados do Flask no HTML.

8. Execução do projeto

```
py -m pip install flask
py app.py
```

Com o servidor ativo, a aplicação fica disponível localmente em `http://127.0.0.1:5000`. O terminal deve permanecer aberto durante a execução.

9. Conclusão

O projeto demonstra a separação entre frontend e backend. HTML e CSS constroem e estilizam a interface, enquanto Python e Flask recebem os dados, executam a lógica de conversão e retornam o resultado ao usuário.